

Documentation for ‘Sensor + Waker Queue Simulation’

Abstract

This document provides a comprehensive overview of the *Sensor + Waker Queue Simulation*, detailing its purpose, system design, constants, data structures, functions, program flow, and expected output. The simulation illustrates the producer-consumer pattern through a sensor that generates random data and a waker that consumes this data at specified intervals. This documentation serves as a guide for understanding the implementation and functionality of the program.

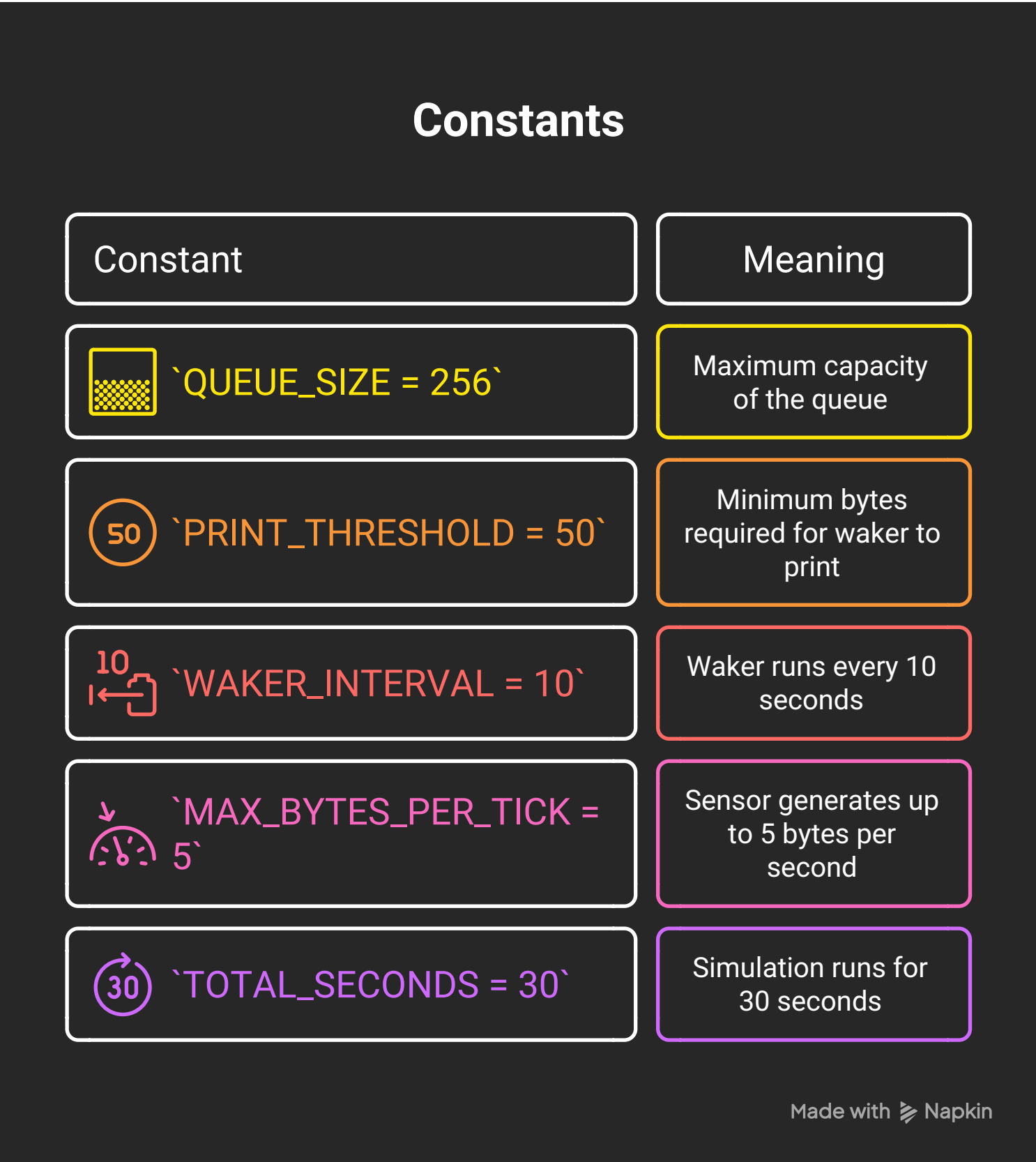
1. Overview

- **Purpose:** Simulate a sensor generating random data and a waker process that periodically consumes data.
- **Context:** Demonstrates how a queue can be used to manage asynchronous data between producer {sensor} and consumer {waker}.

2. System Design

- **Queue (Circular Buffer)**
 - **Fixed size:** 256 bytes.
 - **Stores:** Sensor-generated data.
 - **Operations:** Supports push (add) and pop (remove) operations.
- **Sensor Task**
 - **Frequency:** Runs every second.
 - **Data Generation:** Generates 0–5 random bytes.
 - **Queue Interaction:** Pushes bytes into the queue.
- **Waker Task**
 - **Frequency:** Runs every 10 seconds.
 - **Data Check:** Checks if at least 50 bytes are available.
 - **Data Handling:** If yes → pops 50 bytes, prints them in hex, deletes them from the queue.
 - **Insufficient Data:** If not → prints “Not enough data.”

3.

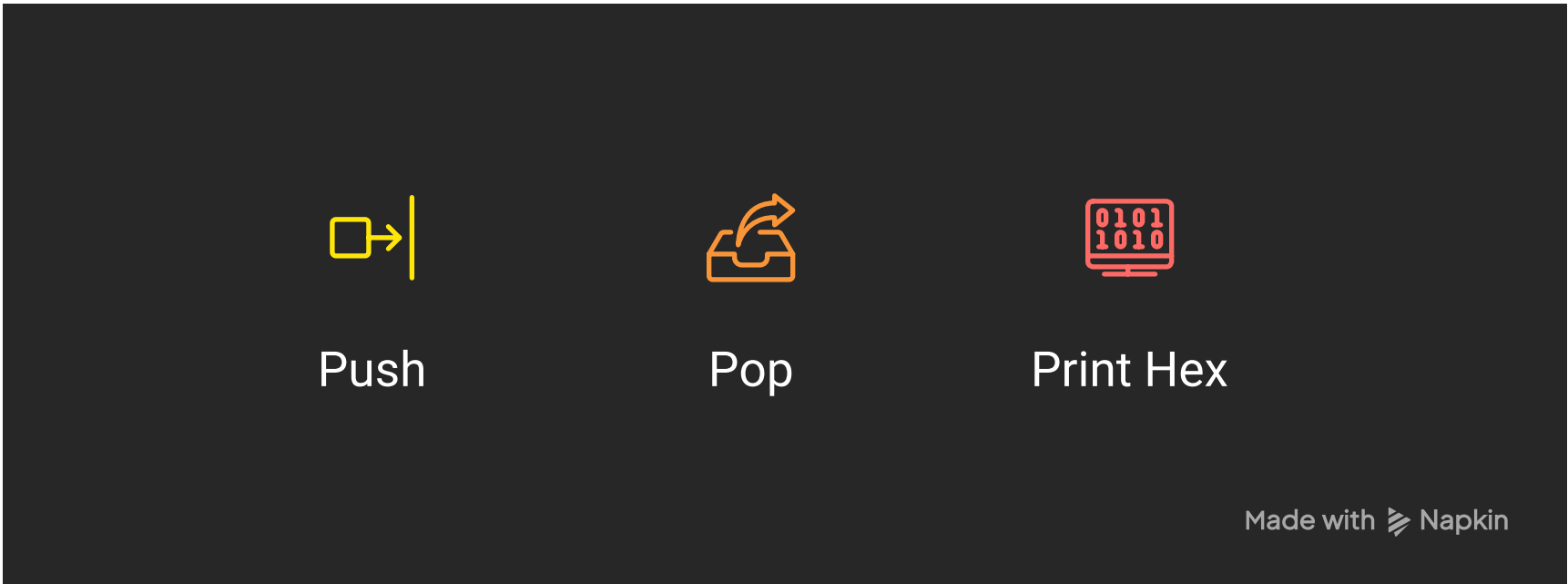


4. Data Structures

```
typedef struct {  
    unsigned char data[QUEUE_SIZE]; // storage  
    int head; // index for next push  
    int tail; // index for next pop  
    int count; // current number of bytes  
} QUEUE;
```

5. Functions

- `push()` → Adds one byte to the queue.
- `pop()` → Removes up to n bytes from the queue.
- `print_hex()` → Displays bytes in hexadecimal format.

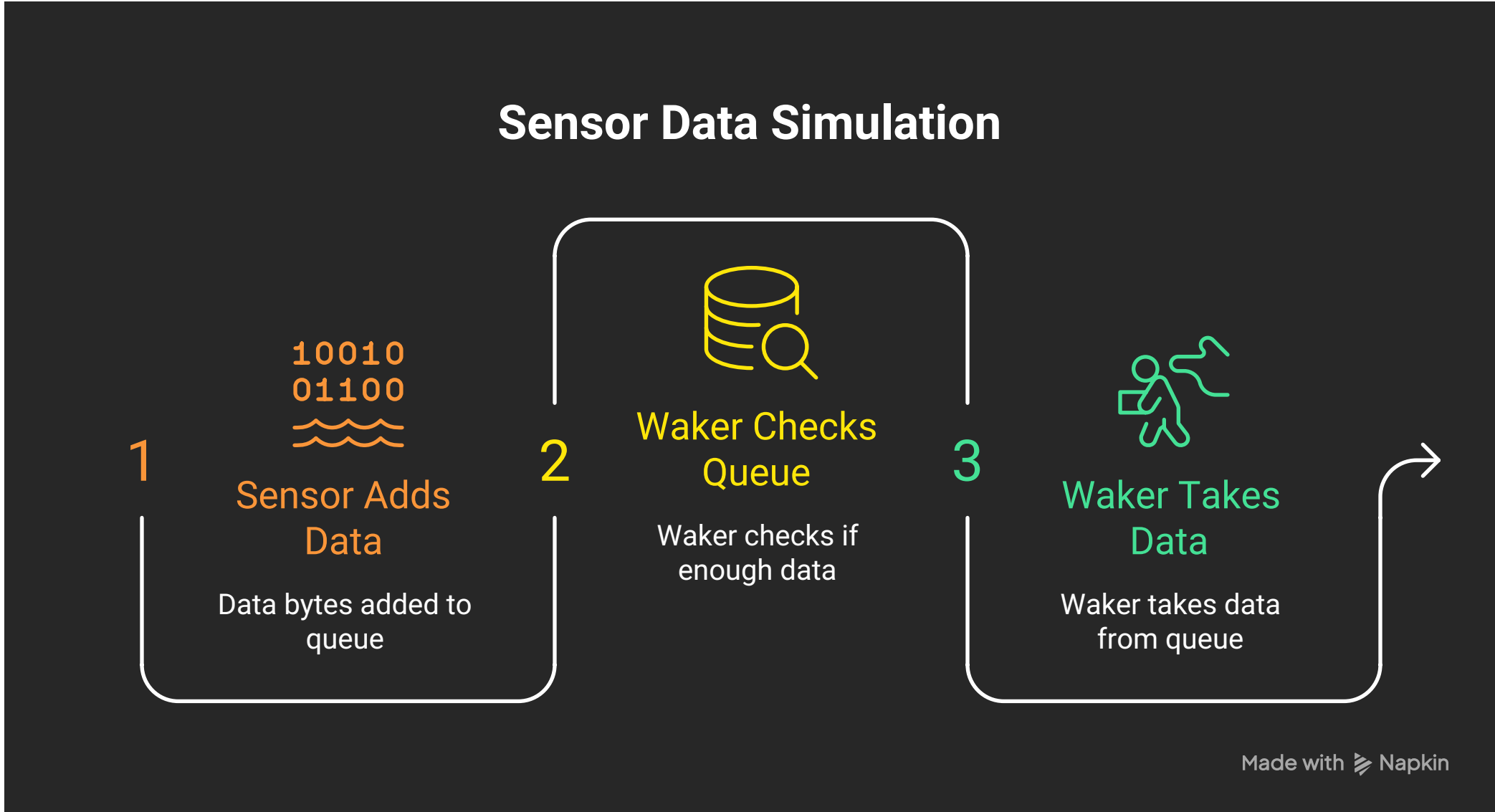


6. Program Flow

1. **Initialize** the queue.
2. **Loop** from second 1 to 30.
 - **Sensor** adds random bytes each second.
 - **Waker** checks every 10 seconds.
 - If ≥ 50 **bytes** → print and remove.
 - Else → log “Not enough.”
3. **End** simulation with a summary message.

7. Sample Output

```
=== Sensor + Waker Simulation ===  
  
Second 1: Sensor added 3 byte(s). Queue now has 3.  
Second 2: Sensor added 5 byte(s). Queue now has 8.  
Second 10: Waker checked, but only 39 in queue. Not enough.  
Second 20: Waker took 50 bytes from queue.  
Data (hex): 3A 7F 12 9C ...  
Second 30: Waker checked, but only 28 in queue. Not enough.  
  
=== Simulation Complete ===
```



8. Key Takeaways

- Demonstrates the producer–consumer pattern using a queue.
- Shows how periodic tasks can interact with shared data.
- Useful for embedded systems or bare-metal simulations.